

2026 Fantasy Football Draft Board — Full Project Writeup

League context: 12-team, superflex (QB, RB, RB, WR, WR, TE, FLEX, SUPERFLEX), full PPR.

1. Restart & Planning

The project started as a redo of an earlier attempt that had two problems: the per-player scoring formula was too complicated (blending multiple factors into one opaque number), and the data had real errors — duplicate rows for traded players (e.g., two "Jakobi Meyers" rows) and players not reflecting their current team (e.g., Jaylen Waddle not showing his trade to Denver).

Decisions made up front:

- Keep the league format the same: 12-team superflex, full PPR.
 - Pivot from **historical stats** to **forward-looking season projections**, since the goal is drafting for the season ahead, not analyzing the past.
 - Draft strategy defined early: use **VORP** to build value tiers and understand positional scarcity, then break ties **within** a tier using judgment factors (age, upside, offensive environment, injury history) — keeping the model simple and putting human judgment where it belongs, at the moment of the actual pick.
-

2. Data Sourcing & Cleaning

Source: FantasyPros' projections tool ([fantasypros.com/nfl/projections/\[pos\].php?week=draft](https://fantasypros.com/nfl/projections/[pos].php?week=draft)), one CSV exported per position (QB, RB, WR, TE), specifically the **full-PPR**, draft/preseason projection set for the 2026 season (confirmed via the `week=draft` parameter and by cross-checking that traded/rookie players showed correct current-team assignments).

One real correction made mid-project: the first CSV export was accidentally **half-PPR**, not full PPR. This was caught, re-exported correctly, and the whole pipeline was rerun — this mattered because full PPR meaningfully raises the value of high-reception players (WRs and pass-catching RBs), shifting rankings, not just totals.

Cleaning steps (Python/pandas, in Google Colab):

1. Loaded each CSV with `skiprows=[1]` to skip a blank formatting row FantasyPros includes.
2. RB and WR files each had **duplicate column names** (YDS/TDS appearing twice — once for rushing, once for receiving) because a player can rush and receive. Pandas auto-suffixes these as `YDS.1/TDS.1`. Renamed explicitly:
 - RB: `ATT`→`RUSH_ATT`, `YDS`→`RUSH_YDS`, `TDS`→`RUSH_TDS`, `YDS.1`→`REC_YDS`, `TDS.1`→`REC_TDS`
 - WR: same idea, reversed order (WR file lists receiving stats first, rushing second)

3. Added a POS column to each of the four dataframes (QB/RB/WR/TE) before merging, so position could be tracked after combining.
4. Merged all four into one combined dataframe with `pd.concat`.
5. Dropped fully blank trailing rows (FantasyPros' export included a couple of blank rows at the bottom of each file — identified because `Player`, `Team`, and `FPTS` were all `NaN`).
6. Converted FPTS to numeric (`pd.to_numeric(errors='coerce')`) and confirmed zero missing values afterward.

Final clean dataset: **526 players** across QB/RB/WR/TE with a validated FPTS (season-long projected PPR points) column.

3. VORP Calculation (Value Over Replacement Player)

Concept: VORP measures how much *more* value a player provides than a readily available "replacement-level" player at the same position — not raw points, but points above what you could get for free/cheap at that position. This makes players **comparable across positions**, which raw points can't do (a QB naturally scores more than a TE; VORP corrects for that).

Step 1 — set replacement-level cutoffs, calibrated for a 12-team **superflex** league (which needs far more startable QBs than a standard league, since two starting lineup slots — QB and SUPERFLEX — can be filled by quarterbacks):

- QB: rank 20 (QB20)
- RB: rank 31 (RB31)
- WR: rank 31 (WR31)
- TE: rank 14 (TE14)

Step 2 — find each position's replacement-level FPTS value: sorted each position by FPTS descending, and took the FPTS of the player sitting exactly at that cutoff rank (e.g., the 20th-ranked QB by projected points). Full-PPR replacement baselines came out to: QB 274.5, RB 185.9, WR 197.3, TE 163.4.

Step 3 — calculate VORP for every player: $VORP = \text{player's FPTS} - \text{their position's replacement-level FPTS}$.

Validation: confirmed the model behaved sensibly — e.g., Josh Allen had the single highest raw FPTS (372.2) of anyone in the dataset, but ranked outside the top 5 by VORP, because in a superflex league the *"replacement-level"* QB is already scoring a lot, shrinking how much "extra" value even an elite QB provides relative to a replacement. Meanwhile top RBs (Jahmyr Gibbs, Bijan Robinson) had the highest VORP overall, reflecting RB's steep positional scarcity.

4. Points Per Game (PPG)

Calculated simply as $FPTS \div 17$ (a full NFL regular season) — this represents each player's **projected average weekly output** for 2026, not a historical number. It's the anchor point that the floor/ceiling calculation builds on top of.

5. Floor and Ceiling

Goal: estimate each player's realistic bad-week and good-week point totals for the *upcoming* season — not a season total, but a single-week range, since that's more actionable for actual lineup decisions.

Method — built in a separate notebook, using real historical data:

1. Took the **top 250 players by VORP** as the player pool for this analysis.
2. Pulled every player's **actual 2025 weekly PPR fantasy point totals** using `nflreadpy` (a free, public NFL stats package) — specifically the `fantasy_points_ppr` column, which `nflreadpy` calculates automatically, filtered to regular-season games only.
3. Matched player names between the two datasets — hit a real snag here: FantasyPros lists suffixes ("Travis Etienne Jr."), `nflreadpy` often doesn't ("Travis Etienne"). Wrote a `clean_name()` function to strip `Jr./Sr./II/III/IV` suffixes before matching, which recovered several mismatches. Ended with 220 of 250 players matched to real 2025 history; the other 30 were genuine 2025 rookies or very limited-usage players with no real 2025 game log.
4. For each matched player, calculated:
 - `games_played`, `avg_2025` (mean weekly PPR score)
 - `p10` and `p90` — their **10th and 90th percentile** weekly scores (not true min/max, to avoid one-off outlier games skewing the estimate)
5. Converted last year's *volatility shape* into *this year's* floor/ceiling:
 - `floor_ratio = p10 / avg_2025`, `ceiling_ratio = p90 / avg_2025` (how far below/above their own average their bad/good weeks tend to run, as a personal ratio)
 - `proj_avg_per_game = this year's FPTS ÷ 17`
 - `proj_floor = proj_avg_per_game × floor_ratio`, `proj_ceiling = proj_avg_per_game × ceiling_ratio`
 - This anchors the **level** to this year's up-to-date projection, while the **shape** (how volatile a player tends to be) comes from real history — so a player's floor/ceiling updates correctly if their situation (team, role) changed.
6. **Rookies/no-history players:** since they have no personal ratio to draw from, filled in using the **average floor/ceiling ratio of real players at their same position**, applied to their own 2026 projection — still analytics-based, not a guess.

Result: every one of the 250 players ended up with a real floor and ceiling, zero missing.

6. The Interactive Draft Board

Built as a self-contained HTML file (usable offline, downloaded and reopened locally), developed iteratively:

Core structure:

- Position tabs: ALL, QB, RB, WR, TE, plus a later-added combined **RB/WR** tab (since that's the bulk of real draft decisions)
- Columns: rank, player name (with team code inline), position, FPTS, VORP, PPG, Floor–Ceiling range
- Sort toggle between VORP and FPTS
- Search by player name
- Team filter dropdown (all 32 real team codes, pulled from the data itself)

Per-player notes: click a row to expand a note field. Originally used Claude's built-in cross-session storage, but since the intended use was **downloading and running the file locally on your own computer**, that storage didn't work outside the chat — switched to the browser's own **local storage**, which persists correctly for a file reopened from the same location in the same browser. Also fixed a save-timing bug (notes were debounced and could be lost if the tab closed before the save fired) by saving immediately on clicking away from a note field, not just after a typing pause.

Drafted tracking: a toggle button per player greys them out and strikes their name when marked drafted, plus a "hide drafted players" checkbox to fully remove them from view. A "Reset Draft" button clears all drafted marks (but never touches notes) for reuse in a new draft.

True rank fix: originally, searching for a player re-numbered them based on their position in the **filtered** results (making anyone look like "#1" when searched alone). Fixed by calculating rank from the full, unfiltered position pool instead.

General notes section: a separate, collapsible free-text box (not tied to any player) for overall draft strategy notes, saved the same way as player notes.

Live Sleeper integration: connected the board to Sleeper's free, public, read-only draft API. Enter your league's draft ID, and the board polls Sleeper's live pick list (originally every 15 seconds, later sped up to every 4 seconds after real mock-draft testing showed lag) and automatically marks players drafted in real time, using the same name-cleaning logic to match Sleeper's naming to the board's. Added a manual "Refresh Now" button since browsers throttle background-tab timers, which could otherwise cause real delays.

Round dividers (ALL tab only) — went through several iterations:

1. First version: dividers based on our own VORP ranking, bucketed in groups of 12 **within each position** — this was flagged as inaccurate (e.g., it implied Matt Stafford would go in "round 2," which isn't realistic).
2. Second version: pulled **real 2026 ADP** (Average Draft Position) data from a live superflex-specific source (DraftBuddy, cross-referencing an FFPC 12-team superflex PPR draft), merged it into the board, and based dividers on that. More accurate, but only covered ~230 of 526 players.
3. **Final version** (your call): simplified back to strict, literal groups of 12 by current sort order (VORP or FPTS) — but with a specific rule that **quarterbacks are removed from round 1 and bumped to the start of round 2**,

reflecting that even in superflex, round 1 is realistically RB/WR territory. Fixed a bug where this bumping caused a false "13th player" to slip into round 1 — the fix ensures round 1 has a **hard cutoff at rank 12** (sometimes fewer than 12 players if a QB was removed from that group), with round 2 absorbing the bumped QB(s) plus enough new players to refill to 12, and rounds 3+ continuing as clean, even groups of 12 from there.

7. The Draft-Round Visualization

Built using `matplotlib`, plotting **average VORP by draft round, per position** — bucketing each position's players into groups of 12 (by VORP rank) and averaging VORP within each group, then plotting all four positions' curves together.

Key honest choice: **left the QB curve uncapped**, showing it crash steeply after round 2–3 rather than truncating the chart to look tidy. This revealed a real, structural insight: in superflex, the startable QB pool (thanks to two teams needing a QB) creates a decent replacement baseline — but past that pool, backup QBs provide almost no value, and the drop-off is a cliff, not a gradual slope, unlike RB/WR/TE. This directly informed the "don't wait too long on QB" nuance in the draft strategy, while also confirming RB has the highest peak value and steepest scarcity, and WR has the flattest, longest tail of any position.

8. Round-by-Round Player Targeting

Attempted to translate the strategy into specific, real player names at each round. First pass used our own VORP-derived "position round" buckets — caught as inaccurate (same root issue as the draft board's round dividers).

Fixed by pulling **real superflex ADP data** and recalculating: took the full 526-player pool, cross-referenced against real ADP round numbers, and rebuilt round-by-round target lists (e.g., confirming which RBs are realistically gone by round 2, which WRs enter at round 3, and — critically — that real QB runs in superflex happen much faster than pure value order suggests, with roughly 8 QBs off the board by the end of round 2 in real drafts, not round 4).

9. Backtesting the Draft Strategy

Goal: test whether the *strategy itself* (not just the value model) would have actually built a good team, using real data from last season.

Data used: a comprehensive real-2025-outcomes dataset (633 players, all four positions) with actual `games_played`, `ppr_per_game`, `ppr_std_dev` (real week-to-week volatility), and `career_avg_ppr` (multi-year track record) — computed real 2025 season totals as `games_played × ppr_per_game`.

Simulation design: a 12-team snake draft (15 rounds), with 11 "opponent" teams always drafting the single highest-value player available, and "your" team following the actual strategy rules (RB-floor-priority rounds 1–2, WR-floor-priority rounds 3–4, wait on QB until ~5–6 are off the board then take ceiling, TE forced late with ceiling priority). After each simulated draft, computed the **optimal starting lineup** (QB, RB, RB, WR, WR, TE, FLEX, SUPERFLEX — 8 starters) from the drafted roster and scored it on real outcomes.

First attempt — flawed, caught and explained honestly: used each player's **actual 2025 final outcome** as the value signal driving **both** teams' draft decisions. This is a "crystal ball" setup — by definition, any rule other than "always take the single highest-scoring player" can only lose value when the drafter already knows the real outcome in advance. Result: the strategy team actually **lost** to the naive best-value team in 8 of 12 simulated draft slots. This was flagged as an unfair test, not a real finding, since it erases the very uncertainty that positional/floor-ceiling strategy exists to manage.

Corrected version — the real test: rebuilt the simulation so both teams draft using only **information realistically knowable before the season** (career_avg_ppr, a rough stand-in for a real preseason projection — deliberately excluding any actual 2025-season data from the decision-making step), while still scoring the final rosters on real 2025 outcomes. This properly separates "what you'd have known on draft day" from "what actually happened."

Result: the strategy team beat the naive best-value team in **all 12 of 12** simulated draft slots — averaging 1920.4 real starter points versus 1705.7 for the naive approach, and beating its own simulated opponents by +175.9 points on average (versus roughly zero for the naive team, as expected when everyone plays that style). The likely mechanism: locking in RB value early protected against the position's steep scarcity risk under real uncertainty, and the floor-then-ceiling sequencing let the team secure a safe foundation before taking calculated upside bets later — exactly the intended logic of the original strategy, now validated against real outcomes under fair, realistic conditions.

Honest limitations, stated plainly

- Floor/ceiling and the backtest both rely on nflreadpy's real 2025 data and a provided real-outcomes CSV — solid, but a single season's worth of variance, not a multi-year distribution.
- Real ADP data covers ~230 of 526 players; deeper bench players don't have real market data.
- The backtest's "opponents" are simplified always-best-value bots, not real human drafters with real tendencies.
- career_avg_ppr is a reasonable stand-in for "preseason-knowable information," but isn't literally last year's real preseason projection (which wasn't accessible).

10. How the Draft Grade Is Measured

Built directly into the interactive draft board, using its live Sleeper connection. Enter your draft slot number and it pulls your actual picks and grades them across **four components**, blended into one letter grade. The panel

always shows the raw numbers behind each component - the math is never hidden behind a single opaque score.

Component 1 - Value vs. Real Market ADP

```
delta = (round you actually drafted them) - (their real ADP round)
```

Positive means you got the player later than the market typically would (a discount); negative means you paid a premium relative to the market. Tags: Steal (+3 or better), Good Value (+2 to +3), Fair (-1 to +2), Slight Reach (-1 to -3), Reach (worse than -3).

A real sign bug was caught and fixed here. The formula was originally written backwards ($\text{adpRound} - \text{actualRound}$), which mislabeled genuine steals as reaches and vice versa - caught by tracing a concrete example by hand.

Component 2 - Positional Scarcity Timing

Recreates the round-by-position VORP decline chart's logic directly from the board's live player data. For a given pick, players at that position are sorted by VORP, and the tool measures:

```
scarcity dropoff = (VORP at this player's rank) - (VORP twelve ranks later)
```

Twelve matches the 12-team league size, mirroring the original chart's bucketing. This answers: *how much value would you have lost at this position by waiting one more round?* Tags: High scarcity (15+ VORP points saved), Moderate (7-15), Low scarcity - could've waited (under 7).

Validated against a real scenario: a user opened a draft with Bijan Robinson then Kenneth Walker III (both RB) and initially received a misleadingly low grade. Recomputed directly from the board's data: Bijan's scarcity dropoff was 118.5 VORP, Kenneth Walker's was 60.2 - versus just 26.3 for a similarly-ranked WR at the same tier. Confirmed the chart's core insight (RB decline is dramatically steeper than WR at the same value tier) was real and had been missing from the original two-component grade.

Component 3 - Team Value

Builds the best possible 8-slot starting lineup (QB, RB, RB, WR, WR, TE, FLEX, SUPERFLEX) from the drafted roster by VORP, and sums it. This folds together two real industry approaches researched for this project - "projected total points" and "value over replacement" - into one fair, position-adjusted number, since VORP already *is* projected points weighed against replacement level, which raw point totals alone don't do fairly across positions (a QB's raw point total isn't comparable to a TE's without that adjustment).

Calibrated against the actual player pool rather than guessed: replacement-level baseline = 0, a realistic solid roster lands around 350, and a theoretical best-possible team tops out near 1,044. Tiers run from "Weak roster value" up through "Elite roster value" using those real anchors.

Component 4 - Roster Balance

Checks real depth against the league's actual starting needs: at least 2 startable (positive-VORP) QBs (superflex demands it), 4 RB, 4 WR, and 1 TE. Score is the average fraction of each target met, capped at 100% per position so no single deep position can mask a punted one. Tags run from "Excellent depth at every position" down to "Poor balance - punted a position too hard."

A second real bug, caught during a later review

Components 3 and 4 were originally computed as if the draft were already finished, even when the user graded their team after only a couple of picks. An unfilled roster slot silently counted as zero value, and unmet depth targets (needing 4 RB, 4 WR, etc., this early) tanked the balance score - meaning **the earlier in a draft you checked your grade, the worse it unfairly looked**, regardless of pick quality. This was traced through concretely: after just two real picks (Bijan Robinson + Kenneth Walker III), a hand-simulation showed the buggy version producing a misleading grade in the C range, even though the individual pick quality (Component 1 and 2 math) supported something much higher.

Fix: Components 3 and 4 now only activate once at least 8 players have been drafted (matching the starting lineup size) - before that point, the blend uses only Components 1 and 2, and the panel clearly labels the other two as "In progress - needs 8+ picks" rather than showing a misleading score. Re-run against the same real two-pick scenario after the fix, the blended grade correctly came out to an A.

A related smaller fix was made at the same time: the code matching a user's picks to their chosen draft slot used strict type equality, which could silently fail to match any picks if Sleeper's API returned that field as a different numeric type than expected. Changed to a safer numeric comparison.

Combining All Four Components

Once at least 8 players are drafted, each component is expressed in comparable "bonus rounds" units and averaged evenly:

```
blended score = (ADP-value delta + scarcity-bonus rounds + team-value bonus + balance bonus) / 4
```

Before 8 picks, only the first two terms are used, divided by 2. The blended score maps to a final letter grade (A+ through F), recalibrated after real mock-draft testing showed the original bands (requiring +1.5 average just for an A) were unrealistically strict - landing close to zero average delta (drafting at fair market value overall) now earns a solid B, matching what real, competent drafting actually looks like.

Known limitations, stated plainly

- Real ADP data covers roughly 230 of 526 players; picks on deeper, less-tracked names show "No ADP data" rather than a fabricated grade.
- Grading works at round-level granularity, not exact overall pick number.
- The scarcity-to-rounds and VORP-to-rounds conversion factors are reasonable, transparent estimates, not formally derived constants.
- Team Value and Roster Balance require a real, evolving roster to mean anything - by design, they stay silent until there's enough of a team to fairly judge.

